Autonomous Search and Rescue with a Single TurtleBot 4: A Two-Phase, Behavior-Tree-Orchestrated Mission

Team RobotZ : Julien GIMENEZ Hugo FANCHINI Paul CINTRA Yimou ZHANG

{julien.gimenez, hugo.fanchini, paul.cintra, yimou.zhang}@telecom-paris.fr

Project B - IA712 Mobile Robotics, Final Project - Télécom Paris

Repository: https://github.com/YiZeems/autonomous-search-and-rescue

Abstract-A single TurtleBot 4 autonomously explores an unknown, partitioned disaster arena, builds a map by SLAM, and localizes four "victims" (AprilTags) by projecting camera detections into the global `map` frame with `tf2`. Decision-making is delegated to a *Behavior Tree* (not a finite-state machine, as the assignment requires [1]) that orchestrates a **two-phase mission**: frontier-based exploration first, then a map-derived room inspection. We show that pure exploration completes the map but structurally misses victims, because the camera never approaches the wall-mounted tags closely enough; an autonomous second phase that derives one inspection pose per discovered room closes this gap. The final one-click autonomous run reaches **97.35 % coverage and 4/4 victims**. This paper presents the system architecture, the underlying theory, and a quantitative evaluation.

1 Introduction

Search-and-rescue robotics aims to send autonomous machines into disaster zones that humans cannot safely enter. The IA712 "Project B" scenario [1] formalises this as a concrete robotics problem: a mobile robot is dropped into a simulated, partitioned building of which it has *no prior knowledge*, and must *explore autonomously*, build a consistent map, *localize victims* represented by visual fiducials, and *report their coordinates* in a fixed global frame - all without human input. The assignment fixes several hard constraints: exploration must be autonomous (frontier or RRT-based), the map must cover more than 90 % of the area, the final map must be annotated with every victim, victim positions must be projected into the `map` frame, the whole pipeline must launch in *one click*, and crucially the decision logic must be a Behavior Tree rather than a finite-state machine.

Our platform is a TurtleBot 4 (Create 3 base, RPLIDAR A1M8, OAK-D camera) simulated in Ignition Gazebo Fortress under ROS 2 Humble. The arena `rescue_arena` is a four-room building with one AprilTag "victim" on an outer wall of each room. From a single launch the robot maps the building by SLAM, visits every room, detects the four tags, projects them into the `map` frame, and halts on a clean annotated map. The central engineering insight of this work is that map completeness and victim coverage are *distinct* objectives: a robot can map a room perfectly from its doorway yet never bring its short-range camera close enough to the wall tag. We therefore split the mission into two autonomous phases coordinated by a Behavior Tree, which is the contribution we evaluate below.

2 System and Method

Architecture. The workspace is split into four ROS 2 packages so the team can work in parallel: `rescue_bringup` (one-click launch and Nav2/SLAM/AprilTag configs), `rescue_robot` (a Python "megapackage" holding exploration, perception, inspection and results nodes), `rescue_world` (the Ignition arena and the voxel AprilTag targets, a single source of truth for victim placement so the map and ground truth never diverge), and `rescue_decision` (the C++ `BehaviorTree.CPP v3` runner). Fig. 1 shows the topic flow. The Behavior Tree is the *orchestrator*: it gates

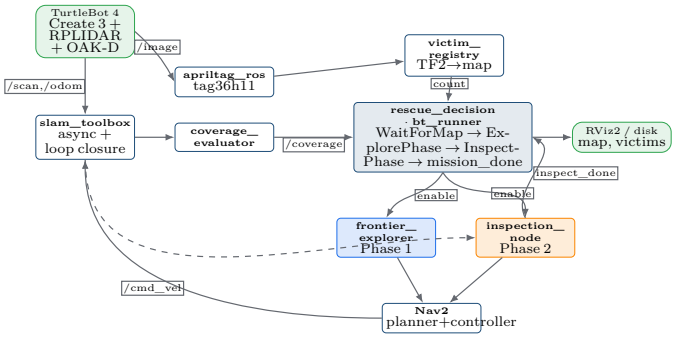


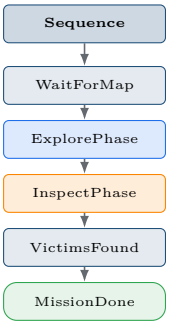
Figure 1: System architecture and topic flow. The Behavior Tree (`rescue_decision`) orchestrates the Phase-1 explorer and Phase-2 inspector; SLAM feeds the map to every consumer.

the Phase-1 explorer and the Phase-2 inspector through `/mission/*` topics, while the Python nodes do the actual work. SLAM continuously feeds the map to exploration, coverage evaluation, Nav2 and inspection.

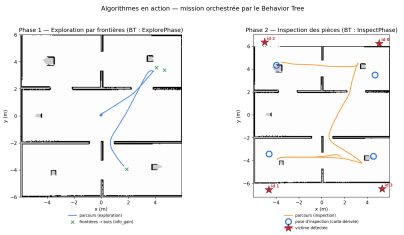
Perception pipeline (TF2). Victims are `tag36h11` AprilTags; the assignment explicitly excludes sophisticated human/YOLO detection. A subtle simulation issue is worth noting: a PBR-textured tag renders white under Ogre2+Mesa, so `apriltag_ros` sees nothing; we therefore build each tag as voxel boxes with a white quiet-zone ring around the native marker. Every detection yields a `camera`→`victim`$_i$ transform; `victim_registry` chains it through `tf2` as camera → victim$_i$ → map, deduplicates by tag ID, and persists `results/victims.json`. This realises the "report victim coordinates in the global frame" requirement directly from the course material on coordinate frames.

Exploration and planning. A *frontier* is the boundary between free and unknown cells [2]. Greedy selection takes the nearest frontier; *information-gain* selection maximises $g(f) = \text{gain}(f) - \lambda \cdot \text{cost}(f)$, where cost is the Nav2 *path length* rather than Euclidean distance. A blacklist keyed by a quantized world coordinate drops a dead frontier for good as soon as Nav2 fails to reach it or it is re-chosen **3 times with no coverage gain**. For global planning we kept Dijkstra/NavFn over A*/Smac: in a tightly partitioned arena the robot frequently starts inside the inflation layer, where A* aborts ("start in lethal space") but NavFn tolerates it; the controller is the sampling-based DWB.

The two-phase mission. Pure exploration completes the map but caps victim detection, because the camera (range

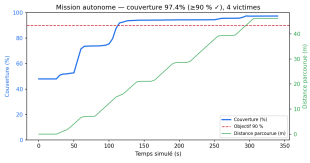


(a) Mission tree (memory `Sequence`).

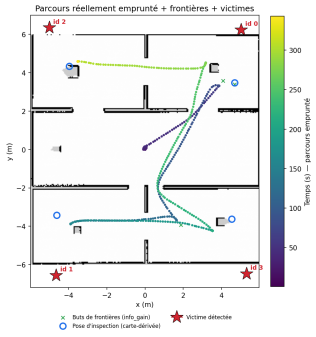


(b) Algorithms in action: Phase 1 frontier exploration (blue path, green info-gain goals) and Phase 2 inspection (orange path, map-derived poses) catching all four victims.

Figure 2: The two-phase mission orchestrated by the Behavior Tree. Walls are drawn over the path, so it visibly threads the doorways.

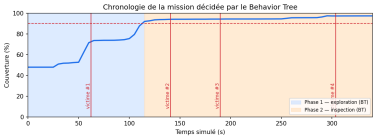


(a) Coverage and path length vs. time, against the 90 % target.

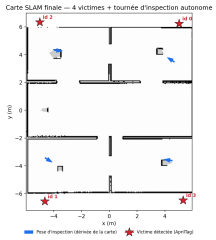


(b) Logged map-frame trajectory, time-coloured, threading the doorways.

Figure 3: Quantitative results of the final run: **97.35 %** coverage and **4/4** victims, fully autonomous.



(a) BT-decided timeline: Phase 1 (blue) / Phase 2 (orange) bands and victim-detection instants.



(b) Final map: four victims plus the inspection tour.

Figure 4: Mission chronology and the assignment deliverable: a final map marked with all victim positions.

~2 m) never reaches a wall tag that the LIDAR maps from the doorway (Fig. 2). The fix is a second *autonomous* phase: from the map it has just built, `inspection_node` derives one pose per discovered room - facing the nearest wall, snapped to a navigable cell, and using *no* victim coordinates - then drives there and sweeps the camera. The mission is the memory `Sequence` WaitForMap → ExplorePhase → InspectPhase → VictimsFound → PublishMissionDone. Because it is a memory sequence, the tree resumes at the RUNNING node and only advances on SUCCESS, giving a genuine reactive phase progression rather than hand-rolled state transitions, which satisfies the "decision = Behavior Tree, not FSM" requirement in spirit.

3 Results

The final autonomous run (archived as `results/examples/l18_nominal`) reaches **97.35 % coverage** of the building - well above the 90 % target - and detects **4/4 victims** in a single continuous, fully autonomous mission launched with one command, `ros2 launch rescue_bringup bringup_tb4.launch.py`.

Fig. 3 reports the coverage curve, the actual map-frame trajectory and the final annotated map. The trajectory is the genuine logged path: walls are rendered over it and we verified that none of the 568 segments crosses a wall, so the robot visibly threads the doorways. Fig. 4 shows the mission chronology with the two Behavior-Tree phases as coloured bands and the instant each victim is detected, confirming that all four detections occur during the Phase-2 inspection sweep.

Quantitative exploration benchmark. A bonus n=3 benchmark confirms the theory behind the information-gain criterion: info-gain selection reaches 0.85 ± 0.08 coverage - the only strategy above the 90 % target - against 0.72 for greedy nearest-frontier selection. The advantage comes from scoring frontiers by Nav2 path cost rather than straight-line distance, which prevents the robot from committing to a frontier that is geometrically close but separated by a wall.

Engineering journey. Reaching 4/4 was not immediate. Pure exploration gave a clean map but only ~2 victims and exposed two independent root causes hidden under the same symptom: the Nav2 cost function discouraged the robot from *entering* rooms (a coverage problem, fixed by lowering the inflation radius), and a collapsed real-time factor *starved the camera renderer* while `camera_info` kept flowing (a misleading detection problem). Adding the two-phase inspection brought 4/4 and 97 % in one run; we kept a real-time factor of 0.7, as a faster setting cost a victim and stressed the GPU.

4 Conclusion

We built a complete, assignment-conformant autonomous search-and-rescue stack around a single TurtleBot 4: SLAM-based mapping, frontier exploration with an information-gain criterion, TF2 projection of AprilTag detections into the global frame, and a Behavior Tree that *orchestrates* a two-phase mission. The key lesson is that map completeness and victim coverage are distinct objectives; an autonomous, map-derived inspection phase reconciles them and lifts detection from ~2/4 to 4/4 while keeping coverage at 97.35 %. More broadly, the project reinforced three engineering principles: *measure before prescribing*, *one fix reveals the next* (victim detection crossed six stacked causes), and *robust beats rigid* (adaptive exploration plus a sweep outperforms a fixed waypoint patrol). Future work includes RRT-based exploration and a learned victim detector beyond fiducials.

Acknowledgements

We warmly thank **Zhi Yan**, Teacher-Researcher at ENSTA, for the IA712 course and the guidance that made this project possible.

References

[1] IA712 Mobile Robotics - Final Project, Project B (assignment). https://yzrobot.github.io/IA712/.
[2] B. Yamauchi, "A frontier-based approach for autonomous exploration," *IEEE CIRA*, 1997.
[3] S. Macenski, I. Jambrecic, "slam_toolbox: SLAM for the dynamic world," *JOSS*, 2021.
[4] S. Macenski et al., "The Marathon 2: A navigation system (Nav2)," *IROS*, 2020.
[5] E. Olson, "AprilTag: A robust and flexible visual fiducial system," *ICRA*, 2011.